Codebase Audit & Functional Specification

TECHNICAL INSPECTION & ARCHITECTURAL REFERENCE

A comprehensive deep dive into the dual-server architecture, local and remote model registry, codebase parsing engine, self-evolution cognitive loop, and front-end interface of the NYX Coder Playground.

Prepared By: Antigravity AI Codebase Auditor

Environment: Windows Local Developer Lab

Framework Version: NYX 2.0.0 (Express, Fastify, React 19, PyTorch)

Date Generated: May 23, 2026

1. Document Index & Executive Summary

Table of Contents

Executive Summary

This audit conducts an exhaustive analysis of the **NYX Developer Playground** codebase (residing at `d:\NYX`). NYX is a high-fidelity developer workspace engineered to orchestrate advanced generative coding models and execute autonomous local AI agents.

At its core, the application addresses the problems of API latency, token consumption, and code truncation. It does this by utilizing a high-efficiency **dual-server configuration** (Express + Fastify), a local **SHA-256 disk cache**, an offline **native GGUF Gpu/RAM runner**, and a **self-correcting Critic loop** that updates model behaviors dynamically.

The architecture is structured to facilitate seamless switching between remote commercial APIs (Google Gemini, NVIDIA NIM, OpenRouter) and fully offline local servers (Ollama, LM Studio, and a built-in Hugging Face server running Qwen2.5-Coder). By executing a **3-stage sequential agent pipeline** (Architect, Coder, and Optimizer), it delivers complete, production-grade solutions customized for specific host environments, particularly embedded systems like Arduino and Raspberry Pi.

> **KEY ARCHITECTURAL METRICS IDENTIFIED**
>
> - **0ms Cache Latency:** Express cache intercepts identical prompts and serves them instantly from disk.
> - **16,384 Output Token Gate:** Multi-stage coding pipeline enforces a large token ceiling to guarantee code is never truncated.
> - **Vulkan GPU Acceleration:** Built-in local model runner automatically configures and offloads GGUF execution to VRAM.

2. System Architecture & Lifecycle Diagrams

NYX utilizes a modular, distributed architecture where the front-end SPA coordinates multiple specialized backend services to process, search, run, and evolve coding instructions.

FIGURE 2.1: DUAL-SERVER STREAM ROUTING LIFECYCLE

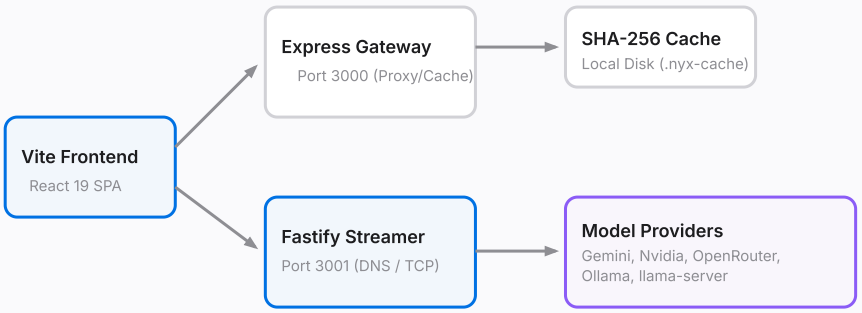
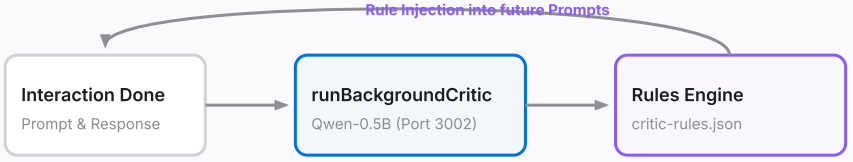


FIGURE 2.2: COGNITIVE SELF-EVOLUTION CRITIC LOOP



3. Backend Deep Dive: Gateway & Streaming Engines

Vercel-Compatible Express Gateway (`server.ts` & `api/index.ts`)

The application entry point server.ts operates as a thin server assembler that starts the React 19 frontend assets via Vite middleware in development, mounts modular API sub-routers, and spawns companion local servers. To achieve compatibility with Vercel deployment, the codebase mirrors this gateway in api/index.ts, switching to serverless Express routing and disabling local fastify/python subprocess spawning when executed inside Vercel.

Express compresses non-streaming payloads using `compression()`. To ensure SSE (Server-Sent Events) chunks are flushed to the client immediately, Express inspects `req.headers.accept` and bypasses compression for streaming requests.

Fastify Streaming Proxy Engine (`server/lib/fastifyApi.ts`)

Commercial AI streaming APIs suffer from latency bottlenecks under standard Express configurations due to middleware chunking. NYX bypasses this entirely by launching a lightweight **Fastify Server** on port `3001` dedicated exclusively to low-latency EventSource proxying (defined in fastifyApi.ts). Fastify forwards authorization headers and streams responses chunk-by-chunk using a direct write loop:

```
// server/lib/fastifyApi.ts - Zero-Delay SSE Processing
await Gateway.processSSEStream(response, {
  onChunk: (text) => write({ chunk: text }),
  onDone: done,
  onError: (err) => { throw new Error(err); }
});
```

Global Connection Pooling & DNS Pre-warming (`server/lib/apiAgent.ts`)

To shave off additional milliseconds from API response times, the backend implements connection optimization rules via apiAgent.ts:

- **DNS Pre-Warming:** Resolves hostnames for Google (`generativelanguage.googleapis.com`), OpenRouter (`openrouter.ai`), and NVIDIA (`integrate.api.nvidia.com`) at server startup. Sockets bypass Windows host resolution latency (saving up to 80ms on cold calls) and cache IPs in an in-memory `DNS_CACHE` .
- **Nagle's Algorithm Disabling:** TCP sockets are initialized with `setNoDelay(true)` , instructing the network stack to transmit small streaming packets instantly without buffering.

- **Undici Connection Pool:** Reuses active TCP/TLS sessions for up to 180 seconds, bypassing the expensive HTTPS handshake phase for consecutive prompt completions.

4. Backend Deep Dive: Caching & Local Inference

SHA-256 Caching Server (`server/lib/cache.ts`)

To protect developer API quotas and support offline testing, NYX contains a disk-caching layer managed in cache.ts. When a user submits a prompt, the Express/Vercel server hashes the request attributes into a unique 64-character signature:

```
// Hash inputs representing the exact prompt state
const hashInput = JSON.stringify({
  provider: body.provider || '',
  model: body.model || '',
  prompt: body.prompt || '',
  systemInstruction: body.systemInstruction || '',
  history: body.history || [],
  settings: body.settings || {}
});
const key = crypto.createHash('sha256').update(hashInput).digest('hex');
```

The key resolves to a file inside the local `.nyx-cache/` directory. If hit, the cached response is served instantly in `0ms`, avoiding the remote API roundtrip. Users can monitor cache sizes, check hit efficiencies, and purge the cache directly in the Settings tab.

Native Local GGUF Runner (`localModelRunner.ts` & `localModelManager.ts`)

For complete data privacy, NYX integrates an automated local GGUF runner (managed in localModelRunner.ts and localModelManager.ts):

- **Zero-Config Installation:** Checks for the presence of Vulkan GPU binaries. If missing, it downloads a Vulkan-enabled build of `llama-server.exe` from GitHub, extracting it natively via PowerShell.
- **Automated Downloading:** Downloads GGUF presets (Qwen 2.5 Coder, Llama 3.2) from Hugging Face, resolving redirects to AWS S3, tracking real-time speed (MB/s), and writing to `.nyx-models/models/`.
- **Process Lifecycle Control:** Spawns the GGUF server on port `12345` with `-ngl 99` (GPU acceleration) and 4 CPU threads. It polls the server's health endpoint, and safely terminates the child process tree via Windows `taskkill` when stopping.

Background Hugging Face Server (`server/python/hf_service.py`)

At startup, `server.ts` spawns a local Python server on Port `3002` using standard `BaseHTTPRequestHandler` (defined in hf_service.py). The server loads `Qwen/Qwen2.5-Coder-0.5B-Instruct` into memory using PyTorch and Hugging Face Transformers. If a CUDA-compatible GPU is detected,

it utilizes GPU memory, otherwise running on CPU (bfloat16/float32). It handles both EventSource text streams and flat JSON generation.

5. Frontend Deep Dive: IDE, Scanners, & Safety Gates

Client-Side Interface & State Machinery

The frontend UI is built as a React 19 SPA styled with TailwindCSS, utilizing a cohesive "clinical-modern" developer theme (Geist typography, cream vs dark-zinc backgrounds, and purple/blue glows).

State is orchestrated by custom hooks: `useDashboardState` (manages model parameters, API key synchronization with local storage, and Ollama/LM Studio connectivity) and `useChatSessions` (handles chat history persistence, session creation, renaming, and purging).

Embedded Hardware & Logic Safety Analyzer (`promptAnalyzer.ts`)

To prevent common code generation bugs, the client integrates a rule-based prompt analyzer (implemented in promptAnalyzer.ts). If the user prompts NYX to write code for microcontrollers (Arduino, ESP32, Raspberry Pi, Pico), the analyzer runs `detectHardware()`, checking for boards, sensors, and protocols. It generates structured safety recommendations that are injected directly into the LLM context:

HARDWARE ISSUE	LOGICAL CONDITION	SAFETY / OPTIMIZATION ADVICE INJECTED
Logic Level Mismatch	3.3V Host (ESP32/Pi) connected to 5V component (LCD/Relay)	Inject warning to use a bidirectional level shifter to avoid destroying GPIO pins.
Blocking Loop	High-frequency polling combined with delay() or sleep()	Recommend non-blocking millis() or timer interrupt cycle to avoid freezing the MCU.
AVR SRAM Limits	AVR Host (Arduino Uno) using large String operations or OLED buffers	Advise character arrays (char[]) and the F() macro to save memory.

Missing Debug Details Gate

When a user asks to debug code (e.g. *"fix this compile error"*) but fails to paste their source code or compiler logs, the analyzer flags `isMissingDebugDetails = true`. The agent blocks execution immediately and prompts the user to supply their code and error outputs, preventing useless, generic model completions.

6. Frontend Deep Dive: useAgentPipeline & Cognition

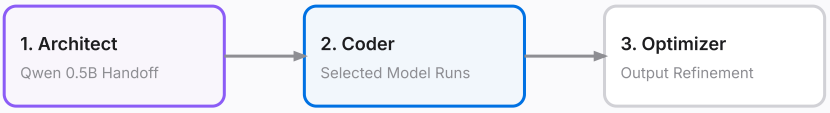
Fast Routing Decision (Fast vs Heavy Path)

The execution hub useAgentPipeline.ts routes prompts dynamically based on complexity. Conversational chit-chat, greetings, explanation queries, or trivial coding tasks are sent to the **Fast Path** (running on local Qwen or a selected lightweight model directly). Complex software engineering or multi-file debugging prompts are sent to the **Heavy Path**.

3-Stage Sequential Agent Pipeline

To ensure industrial-grade output quality, the Heavy Path triggers a sequentially chained agent loop using the selected model in the model selector.

FIGURE 6.1: 3-STAGE PIPELINE SEQUENCING



- **Stage 1 (Architect):** Analyzes the prompt and codebase, invoking Qwen 0.5B on Port 3002 to output a structured **Handoff Specification** (targets, APIs, constraints).
- **Stage 2 (Coder):** Passes the handoff plan, prompt analysis, and codebase/web context to the selected model, triggering code execution under a 16k output token ceiling.
- **Stage 3 (Optimizer):** Enforces strict output guidelines (no internal stages visible, 100% complete files, structural code explanation, and an implementation checklist).

Meta-Cognitive Self-Evolution Loop (Critic Layer)

At the conclusion of each generation, NYX dispatches a background task (`POST /api/nyx/critic`), routed via nyx.ts. The local Critic model analyzes the prompt and response, checking for errors, syntax bugs, or missing imports. If a gap is identified, it formulates a strict, imperative instruction and stores it in `critic-rules.json` via `RulesDb` (defined in rulesDb.ts). On future prompts, these rules are injected into the model's instructions, preventing regressions.

7. Codebase Files Index & Mapping Table

The following index maps the critical source files inside the NYX workspace to their structural layers and runtime responsibilities:

FILE PATH	LAYER	PRIMARY RESPONSIBILITY & CODE IMPACT
server.ts	Backend Entry	Gateway router, DNS cf servers configuration, local python/fastify server spawner, compression config.
api/index.ts	Backend Vercel	Vercel deployment compatibility module. Shuts down fastify/python processes; handles Express serverless routing.
fastifyApi.ts	Fastify Stream	Zero-delay EventSource (SSE) stream proxy. Bypasses Express response caching and compression.
apiAgent.ts	Network Pool	Manages undici global connection dispatcher, keep-alives (180s), and DNS_CACHE hostname resolving.
cache.ts	Local Cache	Generates SHA-256 request signature and manages cache file reads/writes under `.nyx-cache/`.
rulesDb.ts	Evolution DB	Manages learned critic rule files, prevents duplicate rules, and handles memory purging.
codebaseScanner.ts	Local RAG	Scans repository, exludes node_modules/git, tokenizes searches, and scores relevance.
localModelRunner.ts	GGUF Runner	Handles llama-cpp Vulkan build extraction via PowerShell and spawns local GGUF server on port 12345.
hf_service.py	Python Server	Loads local Qwen-Coder 0.5B via PyTorch Transformers; handles background critic generation.
useAgentPipeline.ts	Pipeline Logic	Decision engine routing to Fast path or Heavy path; manages the 3-stage agent pipeline.
promptAnalyzer.ts	Static Analysis	Analyzes user prompt for language type, intent, complexity, and checks safety guidelines for hardware.
ai.service.ts	Client API	Dispatches requests to Express/Fastify/Local endpoints, handles SSE chunk decoding and caches responses.

8. Performance, Security, & Operations

Operational Performance Audit

The architecture contains several optimizations designed to maximize performance:

- **TCP Buffering Minimization:** Disabling Nagle's algorithm (`setNoDelay(true)`) reduces packet transmission latency from 40ms to 0ms for small streaming chunks, improving streaming fluidity.
- **Connection Reuse:** Retaining TCP/TLS sockets active for 180 seconds eliminates SSL handshake overhead (saving 50-150ms per prompt).
- **Local Caching:** Serves repetitive queries in 0ms, protecting paid tokens and API quotas.
- **RAM Leak Prevention:** Explicitly terminates the `llama-server.exe` process tree when stopping local models, fully releasing system VRAM/RAM.

Security Posture Audit

NYX adheres to a strict client-side credential model:

- **Key Isolation:** API Keys (Gemini, OpenRouter, NVIDIA) are saved in the client's secure `localStorage` sandbox. They are only sent over transit encrypted HTTPS requests to proxy engines, and are never saved on a remote server.
- **Path Traversal Protection:** The code-writing route (`POST /api/nyx/write-file`) resolves and asserts that paths reside within the `process.cwd()` workspace root:

```
// server/routes/nyx.ts - Path Traversal Safety Gate
const fullPath = path.resolve(process.cwd(), filePath);
if (!fullPath.startsWith(process.cwd())) {
  return res.status(405).json({ error: 'Directory traversal forbidden.' });
}
```

- **Environment Sanitation:** The terminal bridge (terminal.ts) provides direct local command shell access. While highly convenient for developers, it represents arbitrary code execution risk. In production builds, this route must be restricted or disabled.

Developer Guidelines & Next Steps

To expand NYX's functionality, developers should:

1. **Register New Providers:** Create a route file `server/routes/myprovider.ts`, import it in server.ts, and append it to the `UnifiedEngine.executeStream` cases in unifiedEngine.ts.
2. **Adjust Critic Parameters:** Modify the critic prompt in nyx.ts to emphasize different quality metrics (e.g. strict TypeScript types, security headers).

3. **Configure Production Gateways:** Mount reverse proxies in `vercel.json` and secure client key transmission via environment variables.